

RLdC Core Blueprint

This document outlines the conceptual design and migration plan for a fresh RLdC trading bot implementation.

Objectives

1. **Single Source of Truth:** The bot, the web UI, and the Telegram interface must all read from the same central state, which is synchronised with Binance.
2. **Clear Trade Lifecycle:** Define an explicit state machine for each trade cycle: SCAN → SCORE → VALIDATE → ORDER_CONFIRMED → ORDER_SUBMITTED → ORDER_FILLED → POSITION_OPEN → MONITORING → EXIT_SUBMITTED → EXIT_FILLED → POSITION_CLOSED.
3. **Robust Risk Controls:** Enforce minimum notional values, risk/reward thresholds, duplicate position guards, and market health checks.
4. **Reconciliation Engine:** Continuously reconcile local positions and balances with Binance to correct mismatches.
5. **Modular Architecture:** Separate concerns into scanning, risk, execution, reconciliation, runtime configuration, and integrations.
6. **Testability:** Include unit tests and integration tests to validate every module. Avoid hidden side effects that complicate testing.

Directory Structure

```
RLdC_Core/
├── backend/
│   ├── core/
│   │   ├── engine/      # decision engine and state machine
│   │   ├── execution/   # order submission and tracking
│   │   ├── reconcile/   # reconciliation with Binance
│   │   ├── risk/        # risk parameters and checks
│   │   ├── scanner/     # market scanning and candidate ranking
│   │   └── portfolio/   # account and position models and
│   └── helpers
│       ├── runtime/     # runtime mode and settings
│       ├── integrations/
│       └── binance/     # exchange API client implementation
```

```

|   |   └─ telegram/      # messaging integration
|   └─ ... (other backend components)
├─ api/                  # REST or GraphQL API (placeholder)
├─ overlay/              # overlay/client for streaming data
(placeholder)
├─ web/                  # web front-end (placeholder)
├─ tests/                # unit and integration tests
├─ docs/                 # documentation
└─ README.md             # project introduction and instructions

```

Key Components

Market Scanner (**backend/core/scanner**)

Responsible for building the universe of tradeable symbols, computing scores based on momentum, volatility, volume, and other signals, and validating candidates against risk criteria. It should return a ranked list of executable candidates.

Risk Management (**backend/core/risk**)

Defines constants such as `MIN_ORDER_NOTIONAL_EUR`, `MIN_EXPECTED_NET_PROFIT_PCT`, and `MAX_OPEN_POSITIONS`. Exposes helper functions to evaluate notional value, calculate net expected edge, and enforce duplicate position guards and cooldowns.

Decision Engine (**backend/core/engine**)

Orchestrates the scanning and risk modules to generate a `TradeDecision`. The engine should not submit orders directly; instead, it should produce decisions for the execution layer and maintain state transitions.

Execution (**backend/core/execution**)

Handles the creation, submission, and monitoring of orders. It must confirm `FILLED` status from Binance before transitioning a position to open. Partial fills, rejections, and cancellations must be accounted for. Execution results are recorded for reconciliation.

Portfolio Reconciliation (backend/core/reconcile)

Reconciles local database records with the live account state from Binance. It reconstructs missing positions from trade history and closes orphaned positions. Reconciliation runs should be logged with detailed summaries of differences and actions taken.

Runtime Configuration (backend/core/runtime)

Manages global settings (e.g. MODE=demo or live), account selection, and environment variables. Changing runtime settings must be safe and audited.

Integrations (backend/integrations)

Implement external communication layers:

- **Binance Client:** A robust API wrapper with rate-limiting, error handling, and account snapshot methods.
- **Telegram Bot:** A non-blocking messaging client that sends alerts only after confirmed events (FILLED order, POSITION_OPENED, POSITION_CLOSED), not at signal detection.

Migration Steps

1. **Hotfix Existing Bot:** Apply `fix_rl_dc_ai_nalyzator.sh` to the old project to prevent duplicate buys, enforce FILLED status, and stop deleting closed positions.
2. **Bootstrap New Project:** Run `new_rl_dc_core_skeleton.sh` to generate the RLdC core skeleton. Copy only tested and critical modules from old code (e.g. accounting functions, Binance client) into their appropriate packages.
3. **Rebuild Modules:**
 - Move scanning logic into `market_scanner.py` and rewrite it to rank and validate candidates thoroughly.
 - Flesh out `risk.py` with your real risk parameters and checks.
 - Implement `decision_engine.py` to produce trade decisions from scanner output under risk constraints.
 - Write a new execution module that handles order placement, monitors statuses, and updates positions.
 - Expand `portfolio_reconcile.py` to synchronize local DB with Binance trade history and balances.

4. **Create Tests:** Build test suites under tests/ for scanner ranking, risk checks, decision generation, and reconciliation. Include regression tests covering previously observed failures (duplicate buys, mis-synced positions, phantom fills).
5. **Develop Interfaces:** Gradually reintroduce API, overlay, and web interfaces, ensuring they read from the central state managed by the core. Remove any logic from presentation layers.
6. **Monitor & Iterate:** Before deploying to live trading, run the new core in demo mode alongside the old bot to ensure consistent and correct behaviour. Once validated, migrate fully and decommission legacy code.

This blueprint emphasises gradual migration: stabilise the old system first, then build a robust new core in parallel. Adopt strict testing and reconciliation to prevent previous errors. Focus on a single source of truth and clear state transitions to maintain reliable trading operations.